B1 · Resumen de la ejecución

Dependencias entre Categorías y Productos

Estudiante: Miguel Ángel Tinoco López
Docente: Reyna Barreto Benjamin David
Actividad: Reto 02 · Sesión 8 · Unidad II
Repositorio: github.com/7inox/pharma-frontend
Rama preparada: feature/pruebas-dependencias-tinoco

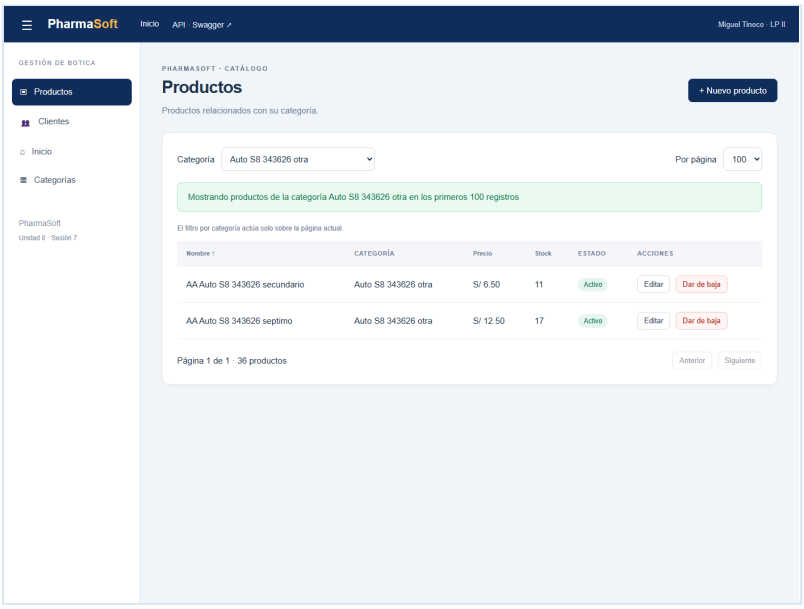
Grupo	Ejecutados	Pasan	Fallan
Altas	8	7	1
Cambios	5	2	3
Bajas	5	5	0
Total	18	14	4

Se ejecutaron los 14 casos base y cuatro propios: A-07 (límites válidos), A-08 (ID negativo), C-05 (cambio a categoría inexistente) y B-05 (categoría con productos activos). La matriz Markdown contiene precondiciones, datos, esperado, obtenido y enlaces a cada captura.

Preparación: categorías exclusivas de prueba; tres activas, una inactiva y ocho productos, incluido uno dado de baja. La API real usa Oracle y responde en localhost:8080. La matriz usó localhost:4201 y la verificación final de la mejora localhost:4203, con un proxy hacia esa misma API.

Método y límite: se probaron la SPA y peticiones HTTP directas. La colección v2.1 contiene las peticiones y respuestas registradas, pero no se ejecutó desde Postman. Importarla y repetirla completa ese requisito. Las pruebas fueron automatizadas con asistencia; el estudiante debe poder reproducirlas en clase.

Parte C · «Ver productos» verificado



El botón navega con queryParams; ProductoList recibe categoriaId, solicita tamanio=100 y selecciona el filtro. Archivos: categoria-list.html y producto-list.ts/.html. La pantalla muestra la nota sobre los primeros 100 registros.

B2 · Fichas de hallazgos

Altas y cambios a categorías inactivas

H-01 · A-04 · La API acepta un alta inválida

Descripción: SPA oculta: true; API 201 creó el producto. Contradice el resultado esperado: SPA no ofrece; API rechaza con 409.

Severidad: alta; permite relaciones inconsistentes entre productos y categorías.

Causa probable: backend/.../service/impl/ProductoServiceImpl.java, create(): busca la categoría por ID, pero no comprueba getEstado().

Corrección propuesta: Validar estado en la capa de servicio antes de guardar y devolver ReglaNegocioException (409). Mantener la exclusión de categorías inactivas en la SPA como ayuda al usuario.



Evidencia: A-04_http.png
Petición:

```
POST http://localhost:4201/api/v1/productos
{"nombre":"AA Auto S8 343626 A04 categoría
inactiva","precio":5.5,"stock":10,"estado":true,"categoriaId":95}
```

HTTP obtenido: 201.

H-02 · C-02 · La API permite cambiar a una categoría inactiva

Descripción: SPA bloquea: true; API 200 guardó categoría inactiva. Contradice el resultado esperado: SPA impide; API rechaza con 409.

Severidad: alta; permite relaciones inconsistentes entre productos y categorías.

Causa probable: ProductoServiceImpl.java, update(): obtiene la categoría, pero no exige que esté activa. ProductoForm.guardar() solo protege el frontend.

Corrección propuesta: Aplicar la misma regla en update() y conservar el producto sin cambios ante 409. Compartir un método que encuentre la categoría y verifique existencia y estado.



Evidencia: C-02_http.png
Petición:

```
PUT http://localhost:4201/api/v1/productos/32
{"nombre":"AA Auto S8 343626 A01
válido","precio":5.5,"stock":10,"estado":true,"categoriaId":95}
```

HTTP obtenido: 200.

```
Categoria categoria = buscarCategoria(dto.getCategoriaId());
if (!Boolean.TRUE.equals(categoria.getEstado())) {
  throw new ReglaNegocioException("La categoría está inactiva.");
}
```

B2 · Fichas de hallazgos

Desactivación y datos desactualizados

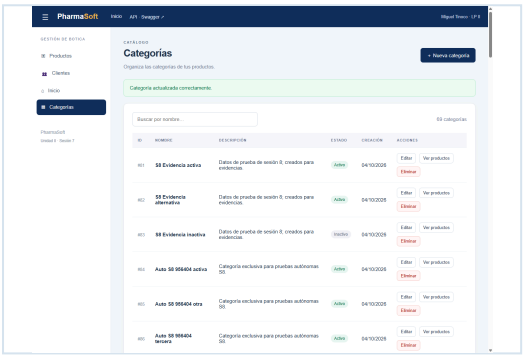
H-03 · C-03 · Se desactiva una categoría con productos activos

Descripción: SPA permitió; PUT 200 dejó producto activo en categoría inactiva. Contradice el resultado esperado: 409; impedir desactivación para preservar productos vendibles.

Severidad: alta; permite relaciones inconsistentes entre productos y categorías.

Causa probable: backend/.../service/impl/CategoriaServiceImpl.java, update(): cambia estado sin consultar si existen productos activos asociados.

Corrección propuesta: Impedir la transición true → false cuando ProductoRepository.existsByCategoriaIdAndEstadoTrue(id) sea true; responder 409. Esta decisión evita dejar productos vendibles bajo una clasificación inactiva.



Evidencia: C-03_spa.png
Petición:

```
PUT http://localhost:4201/api/v1/categorias/96
{"nombre":"Auto S8 343626 ventas","descripcion":"Categoría exclusiva para
pruebas autónomas S8.","estado":false}
```

HTTP obtenido: 200.

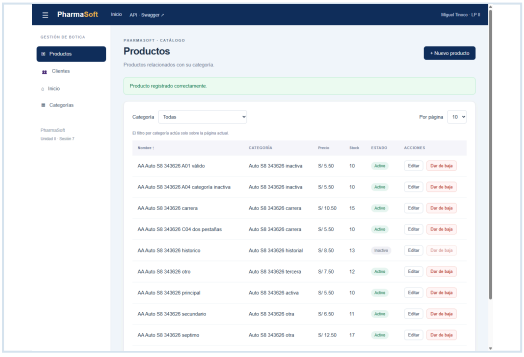
H-04 · C-04 · Dos pestañas eluden la validación de la interfaz

Descripción: Pestaña 2 desactivó; pestaña 1 envió y POST 201 creó el producto. Contradice el resultado esperado: API revalida y rechaza; no crea producto en categoría inactiva.

Severidad: alta; permite relaciones inconsistentes entre productos y categorías.

Causa probable: ProductoForm carga el catálogo al abrir y lo conserva. ProductoServiceImpl.create() tampoco valida estado al procesar el POST.

Corrección propuesta: Revalidar la categoría en la API dentro de la transacción. Para garantizar concurrencia, bloquear la misma fila de categoría al crear/asociar y al desactivarla; así la verificación y la escritura no se intercalan. Mostrar el mensaje 409 en la SPA y recargar las opciones.



Evidencia: C-04_spa.png
Petición:

```
POST http://localhost:4201/api/v1/productos
{"nombre":"AA Auto S8 343626 C04 dos
pestañas","precio":5.5,"stock":10,"estado":true,"categoriaId":97}
```

HTTP obtenido: 201.

Evidencia adicional de C-04: C-04_pestana1.png y C-04_pestana2.png muestran el formulario abierto y la desactivación desde la otra pestaña. El POST de la primera respondió 201. No se corrigió el backend durante esta matriz.

B3 y B4 · Cobertura y recomendación

Dónde están las reglas y cómo mejorar el filtro

B3 · Cobertura de reglas

Regla	Validación actual	Evidencia / alcance
Existencia de categoría	Ambas	La SPA exige una opción cargada; API consulta findById. A-03 y C-05: 404. La SPA no asegura existencia actual.
Categoría activa	Solo SPA	ProductoForm filtra y bloquea; API permite A-04, C-02 y C-04.
Nombre único	Solo API	existsByNombreIgnoreCase / AndIdNot. A-05 devuelve 409; SPA muestra el error.
No eliminar categoría con productos	Solo API	CategoriaServiceImpl.delete usa existsByCategoriaId. B-04/B-05: 409, incluso productos inactivos.
Baja lógica	Ambas	ProductoServiceImpl.delete asigna estado=false; ProductoList recarga y bloquea otra baja. B-01/B-02.
No desactivar categoría con productos activos	Ninguna	Regla adoptada en C-03; el PUT respondió 200.

B4 · Filtro por categoría en el servidor

Actualmente ProductoList.productos filtra resultado.contenido con computed. ProductoService.listar() solo manda pagina, tamanio, ordenarPor y direccion: por eso el filtro no conoce productos de otras páginas. Cargar 100 desde «Ver productos» mejora la consulta, pero mantiene ese límite.

Endpoint: añadir categoriaId opcional y positivo a GET /api/v1/productos. ProductoController y ProductoService.listar deben transmitirlo al repositorio. Mantener todos los estados y los campos de orden permitidos.

```
@RequestParam(required = false) @Positive Long categoriaId

// ProductoRepository; cargar también la categoría
@EntityGraph(attributePaths = "categoria")
Page<Producto> findByCategoriaId(Long categoriaId, Pageable pageable);

// Servicio: paginar después de filtrar, no después de mapear
var page = categoriaId == null ? repo.findAll(pageable)
    : repo.findByCategoriaId(categoriaId, pageable);
```

Frontend: ampliar ProductoService.listar(pagina, tamanio, ordenarPor, direccion, categoriaId?: number | null). Incorporar HttpParams.set('categoriaId', categoriaId) solo si está seleccionado. ProductoList debe pasar ese ID y volver a pagina=0 cuando cambie el filtro.

Usar page.contenido directamente; totalElementos y totalPaginas deben corresponder a la categoría filtrada. Se podrá retirar el límite artificial de 100 y paginar toda la categoría. Conservar queryParams para los enlaces desde Categorías.

B5 · Análisis y entrega

Decisiones sobre integridad y ventas

1. ¿Por qué no basta con ocultar categorías inactivas?

ProductoForm.opciones y guardar() guían al usuario, pero no controlan llamadas externas. A-04 creó un producto con POST 201 y C-02 cambió su categoría con PUT 200. ProductoServiceImpl.create()/update() comprueban existencia, sin estado. La regla debe repetirse en la API porque cualquier cliente puede enviar un categoriaId que la SPA no ofrece.

2. ¿Qué hacer al desactivar una categoría con productos activos?

Elegí impedirlo con 409. Una advertencia deja artículos activos bajo una categoría inválida, y una baja en cascada puede retirar productos vendibles sin que el almacén lo haya decidido. CategoriaServiceImpl.update() debería consultar ProductoRepository.existsByCategoriaIdAndEstadoTrue(id) antes de desactivar. El usuario puede reasignar los productos a una categoría activa o darlos de baja de forma explícita y después desactivar la categoría.

3. ¿Se deben contar también los productos dados de baja?

Sí. CategoriaServiceImpl.delete() usa existsByCategoriaId(), y B-04 respondió 409 aunque el producto estaba inactivo. La baja lógica conserva la relación e información histórica. Eliminar la categoría rompe la integridad referencial o fuerza a perder clasificación en reportes de ventas. Si se necesita ocultarla, conviene cambiar su estado, preservando el registro y la clave foránea.

4. ¿Qué demuestra la prueba de dos pestañas?

C-04 demuestra que validar al cargar el formulario no garantiza el estado al guardar. ProductoForm conservó opciones antiguas y el POST respondió 201 después de desactivar X en otra pestaña. La API es la capa que puede reconsultar y validar justo antes de persistir. ProductoServiceImpl y CategoriaServiceImpl deben coordinar transacciones sobre la misma categoría; un bloqueo de fila o una estrategia de concurrencia equivalente evita una desactivación entre la comprobación y el guardado.

5. ¿Qué cambia en la SPA si el backend corrige A-04?

ProductoForm seguirá filtrando categorías activas, usando ngValue numérico y bloqueando selecciones inválidas. ProductoService seguirá enviando el mismo DTO. Su manejo actual de 409 mediante mensajeError ya permite mostrar la nueva regla; puede añadirse una recarga de categorías tras ese conflicto para actualizar un formulario antiguo. No haría falta modificar rutas, modelos ni el contrato del POST.

Versionamiento y pasos de entrega

Archivos preparados: docs/matriz-pruebas-s8.md, docs/informe-hallazgos-s8.pdf, evidencias nombradas por ID y postman/PharmaSoft-S8-Dependencias.json. Rama: feature/pruebas-dependencias-tinoco; destino del PR: develop.

> Pendientes: aplicar la mejora al proyecto original, repetir las llamadas en Postman, completar commits en días distintos y publicar el PR. Hoy es 04/10/2026; pueden hacerse commits reales el 05 y 06 de octubre antes del plazo 07/10/2026, 23:59. No se falsificaron fechas ni se afirmó que el informe ya está en el aula virtual.

Fuente: s8_lp2_actividad_autonoma_v4jm5dlo8u.pdf, páginas 2-6. Referencias al código: features/productos/pages/producto-form y producto-list; services/producto-service; backend/src/main/java/pe/edu/upeu/Practica/service/impl y repository/ProductoRepository.java.